

2

Управление потоками

В этой главе

- Запуск потоков и различные способы задания кода, исполняемого в новом потоке.
- Ожидать завершения потока или разрешить ему оставаться запущенным?
- Уникальная идентификация потоков.

Итак, вы решили воспользоваться конкурентностью для своего приложения. В частности, задействовать несколько потоков. И что дальше? Как запустить эти потоки, убедиться в их завершении и проследить за ними? Благодаря применению стандартной библиотеки C++ большинство задач управления потоками решить довольно легко, поскольку почти все управление осуществляется с использованием объекта `std::thread`, связанного, как будет показано, с заданным потоком. Для задач с более сложным решением библиотека допускает проявление гибкости, позволяя создавать из базовых строительный блоков нужные вам конструкции.

Сначала в этой главе будут рассмотрены основы: запуск потока, ожидание его завершения или запуск в фоновом режиме. Затем поговорим о передаче дополнительных параметров функции `thread` при ее запуске и передаче права владения потоком от одного объекта `std::thread` другому. И в завершение рассмотрим порядок выбора количества используемых потоков и идентификации конкретных потоков.

2.1. Основы управления потоками

У каждой программы на C++ есть как минимум один поток, запускаемый средой выполнения C++, — поток, выполняющий функцию `main()`. Затем программа может запустить дополнительные потоки, точкой входа в которые служит другая функция. После чего эти потоки и начальный поток выполняются одновременно. Аналогично завершению программы при выходе из `main()` поток завершается при возвращении из функции, указанной в качестве точки входа. Далее будет показано, что при наличии для потока объекта `std::thread` можно дождаться завершения этого потока. Однако сначала его нужно запустить, поэтому посмотрим на запуск потоков.

2.1.1. Запуск потока

В главе 1 было показано, что потоки запускаются созданием объекта `std::thread`, в котором определяется выполняемая в потоке задача. В простейшем случае эта задача представляет собой обычную функцию, которая возвращает `void`-значение, и не принимает никаких параметров. Эта функция выполняется в собственном потоке, пока не вернет управление, после чего поток останавливается. В иных случаях задача может быть функциональным объектом, принимающим дополнительные параметры и выполняющим ряд независимых операций, которые задаются посредством некоей системы обмена сообщениями в ходе его выполнения. Поток останавливается только при получении соответствующего сигнала, опять же через некую систему сообщений. Что бы ни собирался делать поток и откуда бы он ни запускался, его запуск с использованием стандартной библиотеки C++ всегда сводится к созданию объекта `std::thread`:

```
void do_some_work();
std::thread my_thread(do_some_work);
```

Проще, наверное, и быть не может. Конечно, чтобы компилятор мог видеть определение класса `std::thread`, нужно убедиться, что в программу включен заголовок `<thread>`. Как и основная часть содержимого стандартной библиотеки C++, `std::thread` работает с любым вызываемым типом, поэтому конструктору `std::thread` можно также передать экземпляр класса с оператором вызова функции:

```
class background_task
{
public:
    void operator()() const
    {
        do_something();
        do_something_else();
    }
};
background_task f;
std::thread my_thread(f);
```

В данном случае предоставленный функциональный объект копируется в хранилище, принадлежащее вновь созданному потоку выполнения, и вызывается оттуда. Поэтому важно, чтобы копия действовала аналогично оригиналу, иначе результат может не соответствовать ожидаемому.

При передаче функционального объекта в конструктор потока следует избегать так называемого самого неприятного случая синтаксического анализа C++. Если передается временная, а не именованная переменная, синтаксис может быть таким же, как и при объявлении функции. В этом случае компилятор интерпретирует его именно таким образом, а не как определение объекта. Например, код:

```
std::thread my_thread(background_task());
```

объявляет функцию `my_thread`, принимающую единственный параметр наподобие указателя на функцию, не принимающую параметры и возвращающую объект фоновой задачи `background_task`, и вместо запуска нового потока возвращает объект `std::thread`. Такого развития событий можно избежать, присвоив функциональному объекту имя, как было показано ранее, с помощью дополнительных скобок или нового унифицированного синтаксиса инициализации, например:

```
std::thread my_thread((background_task()));
std::thread my_thread{background_task()};
```

В первом примере дополнительные скобки не позволяют интерпретировать код как объявление функции, разрешая объявить `my_thread` переменной типа `std::thread`. Во втором примере используется новый унифицированный синтаксис инициализации с фигурными, а не круглыми скобками, также позволяющий объявить переменную.

Одним из типов вызываемого объекта, позволяющего избежать возникновения рассматриваемой проблемы, является лямбда-выражение. Эта новая особенность, появившаяся в стандарте C++11, допускает создание локальной функции с возможным захватом некоторых локальных переменных, избавляя от необходимости передачи дополнительных аргументов (см. раздел 2.2). Более подробно лямбда-выражения рассматриваются в разделе А.5. С помощью лямбда-выражения предыдущий пример можно записать следующим образом:

```
std::thread my_thread([]{
    do_something();
    do_something_else();
});
```

Научившись запускать поток, нужно принять однозначное решение, ждать ли его завершения (присоединив его — см. подраздел 2.1.2) или пустить его на самотек (путем его отсоединения — см. подраздел 2.1.3). Если не принять решение до уничтожения объекта `std::thread`, то программа завершится (деструктор `std::thread` вызовет метод `std::terminate()`). Поэтому даже при выдаче исключений необходимо убедиться, что поток правильно присоединен или отсоединен. Методы, позволяющие справиться с таким сценарием, рассмотрены в подразделе 2.1.3. Учтите, что

решение нужно принимать до того, как объект `std::thread` будет уничтожен. Сам же поток вполне мог бы завершиться задолго до его присоединения или отсоединения. Если его отсоединить, то при условии, что он все еще выполняется, он и будет выполняться, и этот процесс может продолжаться еще долго и после уничтожения объекта `std::thread`. Выполнение будет прекращено, только когда в конце концов произойдет возвращение из функции потока.

Если не дожидаться завершения потока, необходимо убедиться, что данные, к которым он обращается, будут действительны, пока он не закончит работу с ними. Эта проблема не отличается новизной — даже в однопоточном коде поведение приложения при доступе к объекту после его уничтожения характеризуется неопределенностью, но при использовании потоков вероятность столкнуться с подобными проблемами времени жизни объектов возрастает.

К примеру, похожие проблемы могут возникнуть, если функция потока содержит указатели или ссылки на локальные переменные, а поток при выходе из функции не завершается. Пример такого сценария показан в листинге 2.1.

Листинг 2.1. Функция, возвращающая управление, в то время как в потоке сохраняется обращение к локальным переменным

```
struct func
{
    int& i;
    func(int& i_):i(i_){}
    void operator()()
    {
        for(unsigned j=0;j<1000000;++j)
        {
            do_something(i);
        }
    }
};

void oops()
{
    int some_local_state=0;
    func my_func(some_local_state);
    std::thread my_thread(my_func);
    my_thread.detach();
}
```

Потенциальное обращение к недействительной ссылке

Отсутствие ожидания завершения потока

Возможно, новый поток все еще выполняется

В данном случае новый поток, связанный с `my_thread`, при выходе из `oops`, вероятно, все еще будет выполняться, поскольку, судя по вызову `detach()`, было принято решение не дожидаться его завершения. Если поток все еще выполняется, все идет по сценарию, показанному в табл. 2.1: следующий вызов `do_something(i)` будет обращаться к уже уничтоженной переменной. Все это похоже на обычный однопоточный код, в котором сохранять указатель или ссылку на локальную переменную после выхода из функции непозволительно, но вероятность допустить такую ошибку при использовании многопоточного кода значительно выше, поскольку ее негативные проявления видны не сразу.

Таблица 2.1. Обращение к локальной переменной в отсоединенном потоке после того, как она была уничтожена

Основной поток	Новый поток
Создание <code>my_func</code> со ссылкой на <code>some_local_state</code> Запуск нового потока <code>my_thread</code>	Запуск Вызов <code>func::operator()</code> Запуск <code>func::operator()</code> ; возможен вызов <code>do_something</code> со ссылкой на <code>some_local_state</code>
Отсоединение <code>my_thread</code>	Продолжение выполнения
Уничтожение <code>some_local_state</code> Выход из <code>oops</code>	Продолжение выполнения <code>func::operator()</code> ; возможен вызов <code>do_something</code> со ссылкой на <code>some_local_state</code> => неопределенное поведение

Справиться с подобным сценарием можно одним из распространенных способов: сделать функцию потока автономной и вместо разделения данных скопировать их в поток. При использовании вызываемого объекта для функции потока этот объект копируется в поток, поэтому исходный объект можно тут же уничтожить. Тем не менее следует опасаться объектов, содержащих указатели или ссылки, как, к примеру, в листинге 2.1. В частности, не рекомендуется создавать внутри функции поток, имеющий доступ к ее локальным переменным, если только не будет обеспечено завершение потока до выхода из этой функции.

Или же, *присоединив* поток до выхода из функции, можно убедиться в завершении его выполнения.

2.1.2. Ожидание завершения потока

Дождаться завершения потока можно, вызвав `join()` для связанного экземпляра `std::thread`. Что касается кода в листинге 2.1, то замены вызова `my_thread.detach()` перед закрывающей фигурной скобкой тела функции вызовом `my_thread.join()` было бы достаточно, чтобы гарантировать завершение потока до выхода из функции и, следовательно, до уничтожения локальных переменных. В данном случае это означало бы, что запускать функцию в отдельном потоке нет смысла, поскольку первый поток во время ожидания не будет выполнять никакой полезной работы. Но в реальном коде исходный поток будет либо выполнять полезную работу, либо запускать несколько потоков для выполнения полезной работы, прежде чем входить в режим ожидания их завершения.

Применение метода `join()` — простой и незамысловатый прием: либо вы ждете завершения потока, либо нет. Если нужен более уточненный контроль над ожиданием потока, допустим, с проверкой завершения потока или ожиданием в течение строго определенного времени, придется воспользоваться альтернативными механизмами, например условными переменными и фьючерсами, которые будут рассматриваться в главе 4. Вызов `join()` также приводит к очистке любого хранилища,

связанного с потоком, поэтому объект `std::thread` больше не связан с завершенным потоком. Мало того, он не связан ни с одним потоком. Это означает, что `join()` можно вызвать для конкретного потока только один раз: как только вызван метод `join()`, объект `std::thread` утрачивает возможность присоединения, а метод `joinable()` вернет значение `false`.

2.1.3. Ожидание в исключительных обстоятельствах

Как упоминалось ранее, следует убедиться, что вызов либо `join()`, либо `detach()` был выполнен еще до уничтожения объекта `std::thread`. При отсоединении потока метод `detach()` можно вызвать сразу же после запуска этого потока, так что это не проблема. Но если есть намерение дожидаться завершения потока, то место в коде, где будет выполнен вызов метода `join()`, нужно выбрать с особой тщательностью, поскольку при выдаче исключения после запуска потока, но до вызова `join()` вызов этого метода можно пропустить.

Чтобы избежать завершения приложения при выдаче исключения, необходимо решить, что делать в таком случае. Вообще, если вы намеревались вызвать `join()` при нормальном выполнении программы, вам, чтобы избежать проблем, связанных с временем жизни, необходимо вызвать `join()` и при выдаче исключения. Простой код, выполняющий именно такие действия, показан в листинге 2.2.

Листинг 2.2. Ожидание завершения потока

```
struct func; ← См. определение
void f()      в листинге 2.1
{
    int some_local_state=0;
    func my_func(some_local_state);
    std::thread t(my_func);
    try
    {
        do_something_in_current_thread();
    }
    catch(...)
    {
        t.join();
        throw;
    }
    t.join();
}
```

Код в листинге 2.2 содержит блок `try-catch`, позволяющий гарантировать, что поток с доступом к локальному состоянию завершится до выхода из функции независимо от того, завершается ли функция нормально или из-за выдачи исключения. Использование блоков `try-catch` растягивает код, повышая вероятность получения неправильной области видимости, поэтому данный сценарий далек от идеала. Если есть веские причины обеспечить завершение потока до выхода из функции, скажем, из-за наличия ссылки на другие локальные переменные или

чего-нибудь еще, важно убедиться, что учтены все возможные пути выхода, как нормальные, так и исключительные, и желательно обеспечить для этого простой и лаконичный механизм.

Кроме иных способов, это можно сделать с использованием стандартного способа RAII и, как показано в листинге 2.3, предоставить класс, выполняющий метод `join()` в своем деструкторе. Посмотрите, насколько при этом упрощается функция `f()`.

Листинг 2.3. Использование RAII для ожидания завершения потока

```
class thread_guard
{
    std::thread& t;
public:
    explicit thread_guard(std::thread& t_):
        t(t_)
    {}
    ~thread_guard()
    {
        if(t.joinable())
        {
            t.join();
        }
    }
    thread_guard(thread_guard const&)=delete;
    thread_guard& operator=(thread_guard const&)=delete;
};
struct func; ← См. определение в листинге 2.1
void f()
{
    int some_local_state=0;
    func my_func(some_local_state);
    std::thread t(my_func);
    thread_guard g(t);
    do_something_in_current_thread();
}
```

Когда выполнение текущего потока достигает конца функции `f`, локальные объекты уничтожаются в порядке, обратном порядку их построения. Следовательно, сначала уничтожается объект `g` типа `thread_guard`, а в его деструкторе происходит присоединение к потоку. Это наблюдается даже при завершении выполнения функции, поскольку `do_something_in_current_thread` выдает исключение.

Перед вызовом метода `join()` деструктор `thread_guard` в листинге 2.3 сначала проверяет с помощью метода `joinable()`, является ли объект `std::thread` присоединяемым. Важность этого действия обуславливается тем, что метод `join()` для конкретного потока выполнения можно вызвать только один раз, поэтому, если поток уже был присоединен, его вызов станет ошибкой.

Конструирующий конструктор и операторы копирующего присваивания помечают спецификатором `=delete`, чтобы запретить компилятору автоматически предоставлять их. Копирование или присваивание такого объекта было бы опасным, поскольку тогда время его жизни могло бы превысить время жизни присоединяемого потока.

При объявлении их удаленными любая попытка скопировать объект `thread_guard` будет приводить к выдаче ошибки компиляции. Более подробно удаленные функции рассматриваются в разделе A.2.

Если ждать завершения потока не нужно, проблемы безопасности исключений можно избежать, *отсоединив* этот поток. Тем самым разрывается его связь с объектом `std::thread` и гарантируется, что метод `std::terminate()` не будет вызван при удалении объекта `std::thread`, даже если поток все еще выполняется в фоновом режиме.

2.1.4. Запуск потоков в фоновом режиме

Вызов метода `detach()` для объекта `std::thread` позволяет потоку выполняться в фоновом режиме, непосредственное взаимодействие с ним не требуется. Возможность дождаться завершения этого потока исчезает: если поток отсоединяется, получить ссылающийся на него объект `std::thread` невозможно, поэтому такой поток больше нельзя присоединить. Отсоединенные потоки фактически выполняются в фоновом режиме, владение и управление ими передаются в библиотеку среды выполнения C++, которая гарантирует правильное высвобождение ресурсов, связанных с потоком, при выходе из него.

Отсоединенные потоки часто называются *потоками-демонами* по аналогии с существующей в UNIX концепцией *процесса-демона*, который выполняется в фоновом режиме без какого-либо явного пользовательского интерфейса. Как правило, такие потоки являются весьма продолжительными, работая в течение практически всего времени жизни приложения и выполняя фоновую задачу, например отслеживая состояние файловой системы, удаляя неиспользуемые записи из кэш-памяти объектов или оптимизируя структуры данных. В то же время есть смысл использовать отсоединенный поток, если существует иной механизм определения факта завершения потока или когда нужно запустить задачу и «забыть» о ней.

Как было показано в подразделе 2.1.2, поток отсоединяется путем вызова для объекта `std::thread` компонентной функции `detach()`. После завершения вызова объект `std::thread` больше не связан с фактическим потоком выполнения, поэтому данный поток больше нельзя присоединить:

```
std::thread t(do_background_work);
t.detach();
assert(!t.joinable());
```

Чтобы отсоединить поток от объекта `std::thread`, должен существовать отсоединяемый поток: метод `detach()` нельзя вызывать для объекта `std::thread`, не имеющего связанного с ним потока выполнения. Это требование аналогично тому, которое предъявляется к вызову метода `join()`, и проверку можно провести точно таким же образом — вызывать для объекта `t` типа `std::thread` метод `t.detach()` возможно, только если метод `t.joinable()` вернет значение `true`.

Рассмотрим приложение, к примеру текстовый процессор, позволяющий одновременно редактировать несколько документов. Есть множество способов

справиться с такой задачей как на уровне пользовательского интерфейса, так и на уровне внутренних механизмов. Один из них, получающий сейчас все более широкое распространение, — использование нескольких независимых окон верхнего уровня, по одному для каждого редактируемого документа. Хотя эти окна представляются полностью независимыми и каждое имеет собственные меню, они работают в одном и том же экземпляре приложения. Один из способов справиться с этим за счет внутренних механизмов — запустить все окна редактирования документа в собственных потоках. Каждый поток будет выполнять один и тот же код, но с разными данными, относящимися к редактируемому документу и соответствующими свойствам окна. Следовательно, открытие нового документа потребует запуска нового потока. Поток, обрабатывающий запрос, не станет дожидаться завершения этого другого потока, потому что он работает над другим, независимым документом, что делает новый поток основным кандидатом для запуска в отсоединенном режиме.

В листинге 2.4 показано общее представление кода реализации рассматриваемого подхода.

Листинг 2.4. Отсоединение потока для обработки другого документа

```
void edit_document(std::string const& filename)
{
    open_document_and_display_gui(filename);
    while(!done_editing())
    {
        user_command cmd=get_user_input();
        if(cmd.type==open_new_document)
        {
            std::string const new_name=get_filename_from_user();
            std::thread t(edit_document,new_name);
            t.detach();
        }
        else
        {
            process_user_input(cmd);
        }
    }
}
```

Если пользователь решает открыть новый документ, ему предлагается сделать это, после чего для открытия этого документа запускается новый поток, который затем отсоединяется. Поскольку новый поток выполняет ту же операцию, что и текущий, но в другом файле, той же функцией (`edit_document`) можно воспользоваться повторно, взяв в качестве аргумента новое выбранное имя файла.

В этом примере также показана ситуация, когда есть смысл передать аргументы в функцию, используемую для запуска потока: вместо простой передачи имени функции в конструктор `std::thread` ему передается параметр `filename`. Для этого можно использовать и другие механизмы, например вместо обычной функции с параметрами задействовать функциональный объект с компонентными данными, для чего стандартная библиотека C++ предоставляет очень простой способ.

2.2. Передача аргументов функции потока

В листинге 2.4 было показано, что передача аргументов вызываемому объекту или функции сводится к простой передаче дополнительных аргументов конструктору `std::thread`. Но важно учесть, что по умолчанию аргументы копируются во внутреннее хранилище, где к ним может получить доступ вновь созданный поток выполнения, а затем передаются вызываемому объекту или функции как *r*-значения (*rvalues*), как будто они временные. Так делается, даже если соответствующий параметр в функции ожидает ссылку. Рассмотрим пример:

```
void f(int i, std::string const& s);
std::thread t(f, 3, "hello");
```

В результате создается новый поток выполнения, связанный с `t`, который вызывает функцию `f(3, "hello")`. Обратите внимание: даже если `f` в качестве второго параметра принимает `std::string`, строковый литерал передается как `char const *` и преобразуется в `std::string` только в контексте нового потока. Это становится особенно важным, когда, как показано далее, предоставленный аргумент является указателем на автоматическую переменную:

```
void f(int i, std::string const& s);
void oops(int some_param)
{
    char buffer[1024];
    sprintf(buffer, "%i", some_param);
    std::thread t(f, 3, buffer);
    t.detach();
}
```

Здесь это указатель на буфер локальной переменной, который передается в новый поток. И высока вероятность того, что выход из функции `oops` произойдет, прежде чем буфер будет в новом потоке преобразован в `std::string`, что вызовет неопределенное поведение. Решением является приведение к типу `std::string` перед передачей буфера в конструктор `std::thread`:

```
void f(int i, std::string const& s);
void not_oops(int some_param)
{
    char buffer[1024];
    sprintf(buffer, "%i", some_param);
    std::thread t(f, 3, std::string(buffer));
    t.detach();
}
```

Использование `std::string`
 предотвращает обращение
 к недействительному указателю

В данном случае причиной возникновения проблемы является надежда на подразумеваемое преобразование указателя на буфер в объект `std::string`, ожидаемый в качестве параметра функции, но это преобразование происходит слишком поздно, поскольку конструктор `std::thread` копирует предоставленные значения как есть, не преобразуя в ожидаемый тип аргумента.

Обратный сценарий получить невозможно: объект скопирован и вам нужна не-const-ссылка, поскольку она не скопируется. Все это можно попробовать, если поток обновляет структуру данных, передающую по ссылке, например:

```
void update_data_for_widget(widget_id w, widget_data& data);
void oops_again(widget_id w)
{
    widget_data data;
    std::thread t(update_data_for_widget, w, data);
    display_status();
    t.join();
    process_widget_data(data);
}
```

Хотя `update_data_for_widget` ожидает, что второй параметр будет передан по ссылке, конструктор `std::thread` не знает об этом, он не обращает внимания на типы аргументов, которые ожидает функция, и слепо копирует предоставленные значения. Но внутренний код передает скопированные аргументы в качестве *r*-значений, чтобы работать с типами, предназначенными только для перемещений, и пытается таким образом вызвать `update_data_for_widget` с *r*-значением. Этот код не пройдет компиляцию, так как нельзя передать *r*-значение функции, ожидающей не-const-ссылку. Для тех, кто знаком с `std::bind`, решение будет очевидным: аргументы, которые должны быть ссылками, следует заключать в `std::ref`. В этом случае при изменении вызова потока на:

```
std::thread t(update_data_for_widget, w, std::ref(data));
```

`update_data_for_widget` будет корректно передана ссылка на данные, а не временная копия данных и код успешно пройдет компиляцию.

Если работать с `std::bind` уже приходилось, то в семантике передачи параметров не будет ничего нового, поскольку и операция конструктора `std::thread`, и операция `std::bind` определены в рамках одного и того же механизма. То есть можно, например, передать указатель на компонентную функцию в виде функции при условии, что в качестве первого аргумента предоставлен подходящий указатель на объект:

```
class X
{
public:
    void do_lengthy_work();
};
X my_x;
std::thread t(&X::do_lengthy_work, &my_x);
```

Этот код вызовет `my_x.do_lengthy_work()` в новом потоке, поскольку в качестве указателя на объект предоставляется адрес `my_x`. Такому вызову компонентной функции можно также предоставлять аргументы: третий аргумент конструктора `std::thread` будет первым аргументом компонентной функции и т. д.

Еще один интересный сценарий предоставления аргументов применяется, когда аргументы нельзя скопировать, а можно только *переместить*: данные, содержащиеся в одном объекте, передаются в другой, и исходный объект остается пустым.

Примером может послужить тип `std::unique_ptr`, обеспечивающий автоматическое управление памятью для динамически выделяемых объектов. В одно и то же время на данный объект может указывать только один экземпляр `std::unique_ptr`, и когда этот экземпляр уничтожается, объект, на который он указывал, удаляется. *Перемещающий конструктор* и *перемещающий оператор присваивания* позволяют передавать права владения объектом между экземплярами `std::unique_ptr` (более подробно семантика перемещения рассматривается в подразделе A.1.1. В результате этого исходный объект остается с нулевым указателем. Такое перемещение значений позволяет принимать объекты данного типа в качестве параметров функции или возвращать их из функций. Если исходный объект временный, перемещение выполняется автоматически, но если источником является именованное значение, передача должна быть запрошена напрямую путем вызова метода `std::move()`. В следующем примере показано использование `std::move` для передачи потоку права владения динамическим объектом:

```
void process_big_object(std::unique_ptr<big_object>);
std::unique_ptr<big_object> p(new big_object);
p->prepare_data(42);
std::thread t(process_big_object, std::move(p));
```

Поскольку при вызове конструктора `std::thread` указан метод `std::move(p)`, право владения `big_object` сначала передается внутреннему хранилищу вновь созданного потока, а затем переходит к `process_big_object`.

Некоторые классы в стандартной библиотеке C++ имеют ту же семантику прав владения, что и `std::unique_ptr`, и одним из них является `std::thread`. Хотя экземпляры `std::thread` не владеют, как `std::unique_ptr`, динамическим объектом, они владеют ресурсом: каждый экземпляр отвечает за управление потоком выполнения. Это право собственности разрешается передавать между экземплярами, потому что экземпляры `std::thread` можно переместить, даже если их нельзя скопировать. Тем самым гарантируется, что в любой момент связь с конкретным потоком выполнения будет только у одного объекта, что позволяет программистам передавать это право собственности между объектами.

2.3. Передача права владения потоком

Предположим, что нужно написать функцию, которая создает поток для выполнения в фоновом режиме, но не ожидает его завершения, а передает право собственности на новый поток обратно вызывающей функции, или же сделать обратное: создать поток и передать владение им некоторой функции, которая должна дождаться его завершения. В любом случае нужно передать право собственности из одного места в другое.

Именно здесь и пригодится поддержка перемещения, имеющаяся в `std::thread`. Как говорилось в предыдущем разделе, перемещаться, но не копироваться могут многие типы стандартной библиотеки C++, владеющие ресурсами, например `std::ifstream` и `std::unique_ptr`, и одним из таких типов является `std::thread`.

А это значит, что владение конкретным потоком выполнения можно перемещать между экземплярами `std::thread`, что и показано в следующем примере. В нем создаются два потока выполнения, а права собственности на них передаются между тремя экземплярами `std::thread` — `t1`, `t2` и `t3`:

```
void some_function();
void some_other_function();
std::thread t1(some_function);
std::thread t2=std::move(t1);
t1=std::thread(some_other_function);
std::thread t3;
t3=std::move(t2);
t1=std::move(t3);
```

← Это присваивание приведет к завершению программы!

Сначала запускается новый поток, который связывается с `t1`. Затем при создании `t2` путем вызова `std::move()` для явного перемещения владельца право собственности передается `t2`. В этот момент у `t1` больше нет связанного потока выполнения, и поток, выполняющий `some_function`, теперь связан с `t2`.

Затем запускается новый поток, связанный с временным объектом типа `std::thread`. Последующая передача права собственности в `t1` не требует вызова `std::move()` для его явного перемещения, потому что владелец является временным объектом, а передача права собственности временными объектами подразумевается и выполняется автоматически.

Объект `t3` создан с установками по умолчанию, то есть без связанного с ним потока выполнения. Право собственности на поток, который в данный момент связан с `t2`, передается `t3`, для чего опять применяется явный вызов `std::move()`, поскольку `t2` — именованный объект. После всех этих перемещений `t1` ассоциируется с потоком, выполняющим функцию `some_other_function`, у `t2` связанного потока нет, а `t3` связан с потоком, выполняющим `some_function`.

Последнее перемещение переносит владение потоком, выполняющим `some_function`, обратно в `t1`, откуда он начинался. Но в данном случае объект `t1` уже имел связанный с ним поток, который выполнял `some_other_function`, поэтому для завершения программы вызывается метод `std::terminate()`.

Это делается для обеспечения согласованности с деструктором `std::thread`. В подразделе 2.1.1 было показано, что нужно ожидать явного завершения или отсоединения потока перед уничтожением, то же самое относится и к присваиванию: нельзя просто удалить поток, присвоив управляющему им объекту `std::thread` новое значение.

Реализованная в `std::thread` поддержка перемещения означает, что право собственности можно легко передать из функции (листинг 2.5).

Листинг 2.5. Возвращение `std::thread` из функции

```
std::thread f()
{
    void some_function();
    return std::thread(some_function);
}
std::thread g()
```

```
{
    void some_other_function(int);
    std::thread t(some_other_function,42);
    return t;
}
```

Аналогично если владение должно быть передано функции, то, как показано далее, она может принять экземпляр `std::thread` по значению в качестве одного из параметров:

```
void f(std::thread t);
void g()
{
    void some_function();
    f(std::thread(some_function));
    std::thread t(some_function);
    f(std::move(t));
}
```

Одно из преимуществ поддержки перемещения `std::thread` — возможность построить класс `thread_guard`, показанный в листинге 2.3, и принудить его к владению потоком. Это позволит избежать неприятных последствий в том случае, если объект `thread_guard` переживет поток, на который ссылается. Это также означает, что никто другой не может присоединить или отсоединить поток после того, как право собственности на него передано объекту. Поскольку цель всего этого — в первую очередь обеспечить завершение потоков до выхода из области видимости, я назвал данный класс `scoped_thread`. Реализация и простой пример показаны в листинге 2.6.

Листинг 2.6. Класс `scoped_thread` и пример его применения

```
class scoped_thread
{
    std::thread t;
public:
    explicit scoped_thread(std::thread t_):
        t(std::move(t_))
    {
        if(!t.joinable())
            throw std::logic_error("No thread");
    }
    ~scoped_thread()
    {
        t.join();
    }
    scoped_thread(scoped_thread const&)=delete;
    scoped_thread& operator=(scoped_thread const&)=delete;
};

struct func; ←—— См. листинг 2.1
void f()
{
    int some_local_state;
    scoped_thread t{std::thread(func(some_local_state))};
    do_something_in_current_thread();
}
```


Код примера похож на приведенный в листинге 2.3, но новый поток вместо создания для него отдельной именованной переменной передается непосредственно в `scoped_thread`. Когда исходный поток достигает конца `f`, объект `scoped_thread` уничтожается, а затем этот поток присоединяется к потоку, переданному конструктору. В ходе работы с классом `thread_guard` из листинга 2.3 деструктор должен был проверить, что поток все еще является присоединяемым, а здесь это можно сделать в конструкторе, выдав в случае отрицательного результата исключение.

Одно из предложений для C++17 касалось класса `joining_thread`, который был бы похож на `std::thread`, за исключением того, что автоматически включался бы в деструктор так же, как это делает `scoped_thread`. Но к сожалению в Комитете прийти не удалось, и это предложение не было включено в стандарт (хотя в виде класса `std::jthread` оно все еще актуально для перспективного стандарта C++20), но его относительно несложно создать. Одна из возможных реализаций показана в листинге 2.7.

Листинг 2.7. Класс `joining_thread`

```
class joining_thread
{
    std::thread t;
public:
    joining_thread() noexcept=default;
    template<typename Callable,typename ... Args>
    explicit joining_thread(Callable&& func,Args&& ... args):
        t(std::forward<Callable>(func),std::forward<Args>(args)...)
    {}
    explicit joining_thread(std::thread t_) noexcept:
        t(std::move(t_))
    {}
    joining_thread(joining_thread&& other) noexcept:
        t(std::move(other.t))
    {}
    joining_thread& operator=(joining_thread&& other) noexcept
    {
        if(joinable())
            join();
        t=std::move(other.t);
        return *this;
    }
    joining_thread& operator=(std::thread other) noexcept
    {
        if(joinable())
            join();
        t=std::move(other);
        return *this;
    }
    ~joining_thread() noexcept
    {
        if(joinable())
            join();
    }
};
```

```

}
void swap(joining_thread& other) noexcept
{
    t.swap(other.t);
}
std::thread::id get_id() const noexcept{
    return t.get_id();
}
bool joinable() const noexcept
{
    return t.joinable();
}
void join()
{
    t.join();
}
void detach()
{
    t.detach();
}
std::thread& as_thread() noexcept
{
    return t;
}
const std::thread& as_thread() const noexcept
{
    return t;
}
};

```

Поддержка перемещения в `std::thread` допускает также использование контейнеров объектов `std::thread`, если эти контейнеры поддерживают перемещения, например применение обновленного `std::vector<>`. Это позволяет создавать код из листинга 2.8 — он порождает несколько потоков, а затем ожидает их завершения.

Листинг 2.8. Порождение нескольких потоков и ожидание их завершения

```

void do_work(unsigned id);
void f()
{
    std::vector<std::thread> threads;
    for(unsigned i=0;i<20;++i)
    {
        threads.emplace_back(do_work,i); ← Порождение потоков
    }
    for(auto& entry: threads) ← Поочередный вызов join()
        entry.join();          для каждого потока
}

```

Если потоки, как часто требуется, используются для разделения работы алгоритма, то перед возвратом управления вызывающему коду все они должны быть

завершены. Простая структура листинга 2.8 подразумевает, что работа, выполняемая потоками, автономна, а результатом их операций являются только побочные эффекты для совместно используемых данных. Если бы функция `f()` должна была вернуть вызывающей программе значение, зависящее от результатов операций, выполненных в потоках, то при такой организации получить это значение можно было бы только после анализа разделяемых данных по завершении всех потоков. Альтернативные схемы передачи результатов операций между потоками рассматриваются в главе 4.

Помещение объектов `std::thread` в `std::vector` — шаг к автоматизации управления этими потоками: вместо создания для них отдельных переменных и непосредственного соединения с ними можно рассматривать их как группу. Или пойти дальше, создав динамическое количество потоков, определяемое в ходе выполнения программы, а не фиксированное, как в листинге 2.8.

2.4. Выбор количества потоков в ходе выполнения программы

Одна из функций стандартной библиотеки C++, помогающая решить данную задачу, — `std::thread::hardware_concurrency()`. Она возвращает то количество потоков, которые действительно могут работать одновременно в ходе выполнения программы. Например, в многоядерной системе оно может быть увязано с числом ядер центрального процессора. Функция дает всего лишь подсказку и может вернуть 0, если информация недоступна, но ее данные способны принести пользу при разбиении задачи на потоки.

В листинге 2.9 показана простая реализация параллельной версии `std::accumulate`. В реальном коде потребуется, скорее всего, не самостоятельно создавать эту функцию, а использовать параллельную версию `std::reduce`, рассматриваемую в главе 10, но данная реализация служит иллюстрацией основной идеи. Она распределяет работу между потоками с минимальным количеством элементов на каждый из них, чтобы избежать высоких издержек на слишком большое количество потоков. Заметьте, здесь предполагается, что ни одна из операций не выдаст исключение, даже если они возможны. Конструктор `std::thread`, к примеру, будет выдавать исключение, если не сможет запустить новый поток выполнения. В этом простом примере обработка исключений в подобном алгоритме не предусмотрена (она рассматривается в главе 8).

Листинг 2.9. Простейшая параллельная версия `std::accumulate`

```
template<typename Iterator,typename T>
struct accumulate_block
{
    void operator()(Iterator first,Iterator last,T& result)
    {
        result=std::accumulate(first,last,result);
    }
}
```

```

};
template<typename Iterator,typename T>
T parallel_accumulate(Iterator first,Iterator last,T init)
{
    unsigned long const length=std::distance(first,last);
    if(!length)
        return init;
    unsigned long const min_per_thread=25;
    unsigned long const max_threads=
        (length+min_per_thread-1)/min_per_thread;
    unsigned long const hardware_threads=
        std::thread::hardware_concurrency();
    unsigned long const num_threads=
        std::min(hardware_threads!=0?hardware_threads:2,max_threads);
    unsigned long const block_size=length/num_threads;
    std::vector<T> results(num_threads);
    std::vector<std::thread> threads(num_threads-1);
    Iterator block_start=first;
    for(unsigned long i=0;i<(num_threads-1);++i)
    {
        Iterator block_end=block_start;
        std::advance(block_end,block_size);
        threads[i]=std::thread(
            accumulate_block<Iterator,T>(),
            block_start,block_end,std::ref(results[i]));
        block_start=block_end;
    }
    accumulate_block<Iterator,T>()(
        block_start,last,results[num_threads-1]);

    for(auto& entry: threads)
        entry.join();
    return std::accumulate(results.begin(),results.end(),init);
}

```

Несмотря на большой объем кода, в этой функции нет ничего сложного. Если входной диапазон пуст, возвращается исходное значение, указанное в качестве значения параметра `init`. В противном случае в диапазоне есть хотя бы один элемент, поэтому, чтобы получить максимальное количество потоков, количество обрабатываемых элементов можно разделить на минимальный размер блока. Это сделано для того, чтобы не создавать 32 потока на 32-ядерном компьютере, если в диапазоне только пять значений.

Количество запускаемых потоков — это минимальное значение из полученного в результате расчетов максимума и количества аппаратных потоков. Не следует запускать больше потоков, чем может поддерживать имеющееся оборудование (приводя к появлению так называемого *превышения лимитов*), поскольку переключение контекста при превышении количества потоков повлечет за собой снижение производительности. Если при вызове `std::thread::hardware_concurrency()` возвращается 0, количество выбирается по вашему усмотрению, в данном случае я выбрал 2. Запускать слишком много потоков нежелательно, поскольку на одноядерном компьютере

это замедлит работу, но нежелательно запускать и слишком мало потоков, потому что будет не реализована возможность использования доступной конкурентности.

Количество элементов, обрабатываемых каждым потоком, равно длине диапазона, разделенной на количество потоков. Если число не делится без остатка, не стоит беспокоиться, с этой ситуацией мы справимся чуть позже.

После определения количества существующих потоков можно создать `std::vector<T>` для промежуточных результатов и `std::vector<std::thread>` для потоков. Следует отметить, что нужно запустить на один поток меньше, чем значение `num_threads`, поскольку один поток уже имеется.

Запуск потоков представлен простым циклом: итератор `block_end` сдвигается в конец текущего блока, и для накопления результатов для данного блока запускается новый поток. Окончание текущего блока служит началом следующего.

После того как вы запустили все потоки, данный поток может обработать последний блок. Здесь учитывается любое деление с остатком: известно, что за окончанием последнего блока ничего быть не должно, и количество элементов в этом блоке уже не имеет значения.

После того как накоплены результаты для последнего блока, можно дождаться завершения всех потоков, созданных, как в листинге 2.8, с помощью `std::for_each`, а затем сложить результаты с финальным вызовом `std::accumulate`.

Прежде чем распрощаться с этим примером, стоит отметить, что там, где оператор сложения для типа `T` не является ассоциативным (например, для `float` или `double`), результаты этого `parallel_accumulate` могут отличаться от результатов `std::accumulate` из-за разбиения диапазона на блоки. Кроме того, к итераторам предъявляются несколько более жесткие требования: они должны быть по меньшей мере *однонаправленными*, `std::collectulate` может работать с *однопроходными входными итераторами*, а `T` должен *допускать конструирование по умолчанию*, чтобы можно было создать вектор результатов `results`. Изменения требований такого рода общие для параллельных алгоритмов: чтобы стать параллельными, они по своей природе должны быть другими, что отражается на результатах и требованиях. Более подробно реализация параллельных алгоритмов рассматривается в главе 8, а глава 10 посвящена стандартным средствам, имеющимся в C++17 (эквивалентом рассмотренного здесь `parallel_accumulate` является параллельная форма `std::reduce`). Стоит также отметить, что невозможность возвращать значение непосредственно из потока вынуждает передавать ссылку на соответствующую запись в векторе результатов `results`. Альтернативные способы возвращения результатов из потоков с помощью фьючерсов рассматриваются в главе 4.

В данном случае вся информация, необходимая каждому потоку, включая место, где будет храниться результат его вычисления, была передана при запуске потока. Но так бывает не всегда, и порой в процессе работы необходимо каким-то образом идентифицировать потоки. Можно, как в листинге 2.8, передать идентификационный номер в виде значения `i`, но если функция, нуждающаяся в идентификаторе, находится в стеке вызовов глубже на несколько уровней и может быть вызвана из любого потока, то делать это таким образом неудобно. Эта необходимость была нами предусмотрена при разработке стандартной библиотеки C++, поэтому уникальный идентификатор имеется у каждого потока.

2.5. Идентификация потоков

Идентификаторы потоков относятся к типу `std::thread::id`, и их можно получить двумя способами. Во-первых, идентификатор потока можно получить из связанного с ним объекта `std::thread`, вызвав компонентную функцию `get_id()`. Если у объекта `std::thread` нет связанного с ним потока выполнения, вызов `get_id()` возвращает созданный по умолчанию объект `std::thread::id`, который означает «не поток». Во-вторых, идентификатор можно получить, если вызвать функцию `std::this_thread::get_id()`, которая также определена в заголовке `<thread>`.

Объекты типа `std::thread::id` можно свободно копировать и сравнивать, иначе в качестве идентификаторов от них не было бы никакой пользы. Если два объекта типа `std::thread::id` равны, значит, они представляют один и тот же поток или оба имеют значение «не поток». Если два объекта не равны, они представляют разные потоки или один представляет поток, а другой имеет значение «не поток».

Стандартная библиотека C++ не ограничивает возможности лишь проверкой того, одинаковы идентификаторы потоков или нет: объекты типа `std::thread::id` предлагают полный набор операторов сравнения, предоставляющих весь спектр возможностей по упорядочению всех отличающихся друг от друга значений. Поэтому их можно использовать в качестве ключей в ассоциативных контейнерах, сортировать или сравнивать любым другим способом, который программист сочтет нужным. Операторы сравнения обеспечивают полноценное упорядочение всех неравных значений `std::thread::id`, поэтому они ведут себя интуитивно предсказуемым образом: если `a < b` и `b < c`, то `a < c` и т. д. Стандартная библиотека также предоставляет класс `std::hash<std::thread::id>`, поэтому значения типа `std::thread::id` можно использовать в качестве ключей в новых неупорядоченных ассоциативных контейнерах.

Экземпляры `std::thread::id` часто применяются для проверки необходимости выполнения потоком какой-либо операции. Например, если потоки, как в листинге 2.9, используются для разделения работы, то исходному потоку, запустившему другие потоки, может потребоваться выполнить свою работу в середине алгоритма немного иначе. В данном случае он может сохранить результат `std::this_thread::get_id()` перед запуском других потоков, а затем основная часть алгоритма, которая является общей для всех потоков, может проверить собственный идентификатор потока на предмет соответствия сохраненному значению:

```
std::thread::id master_thread;
void some_core_part_of_algorithm()
{
    if(std::this_thread::get_id()==master_thread)
    {
        do_master_thread_work();
    }
    do_common_work();
}
```

Как вариант, `std::thread::id` текущего потока может храниться в структуре данных как часть операции. Затем последующие операции с той же структурой

данных могут сравнить сохраненный идентификатор на предмет соответствия идентификатору потока, выполняющего операцию, чтобы определить, какие операции разрешены или необходимы.

Точно так же идентификаторы потока можно применять в качестве ключей в ассоциативных контейнерах, где конкретные данные должны быть связаны с потоком, а альтернативные варианты, такие как локальное хранилище потока, для этого не подходят. Такой контейнер может, к примеру, использоваться управляющим потоком для хранения информации о каждом из потоков, находящихся под его управлением, или для передачи информации между ними.

Суть в том, что в большинстве случаев `std::thread::id` в качестве общего идентификатора для потока вполне достаточно, альтернативный вариант может понадобиться, только если идентификатор должен иметь связанное с ним семантическое значение, например, должен быть индексом в массиве. Экземпляр `std::thread::id` можно даже записать в выходной поток, например в `std::cout`:

```
std::cout<<std::this_thread::get_id();
```

Конкретный вывод полностью зависит от реализации, стандарт лишь гарантирует, что идентификаторы потоков, определяемые при сравнении как одинаковые, выдают одинаковый вывод, а неодинаковые — различный. Следовательно, в первую очередь это пригодится при отладке и ведении учетных записей, но значения не несут никакой семантической нагрузки, и сделать на их основе дополнительные выводы невозможно.

Резюме

В этой главе мы рассмотрели основы управления потоками с применением стандартной библиотеки C++: запуск потоков, ожидание их завершения и работу без такого ожидания, если нужно, чтобы они выполнялись в фоновом режиме. Было также показано, как при запуске потока передавать аргументы в функцию потока, а ответственность за управление потоком — из одной части кода в другую и как группы потоков можно использовать для разделения работы. И наконец, мы обсудили механизм идентификации потоков, позволяющий ассоциировать с потоком данные или поведение в тех случаях, когда использовать другие средства неудобно. Конечно же, многое можно сделать и с полностью независимыми потоками, каждый из которых будет работать с отдельными данными, но порой желательно, чтобы запущенные потоки могли работать с общими для них данными. В главе 3 будут обсуждаться вопросы, связанные с разделением данных непосредственно между потоками, а в главе 4 — более общие вопросы, связанные с синхронизацией операций с совместно используемыми данными и без них.